Black-box deterministic expectation estimation for constant-width ROBPs from hitting sets

Kuan Cheng, William M. Hoza

doi:10.4086/toc.2022.v018a021; CC BY 3.0 Unported

Editorial note. Editable author source from officialToC SourceZIP under articleCCBY3.0, with full selectedSection3 proof and necessary introductory definitions. Old same-sourceTheorem2.1 is not selected, repeated or counted; its one analogous-statement reference is an accurate external originalTheorem2.1 location. Original author formulas/indices/widths/space bounds retained without mathematical correction or added hypotheses. Publisherclass/template dependencies replaced by minimal presentation macros; no input/include or figures needed. Original section/theorem numbering restored, explicit source-local correctness and rounding/amplification included. Bibliography publisher formatting macros expanded to ordinary names/links; no bibliography package required. All production/validation exclusivelyYupeng; root owns compilation.

Allowed standard prerequisites (editorial).

- Read-once branching programs, hitting sets, oracle space models and standard hitting-set construction. OriginalSections1.1/1.3/1.5 definitions and fullSection3; publishedHozaZuckerman result cited only for uncounted unconditional instance

Model and definitions (original Section1)

A width-w length-n read-once branching program (ROBP) is a directed graph consisting of $n+1$ layers with w vertices per layer. There is a designated "start vertex" v_{start} in the first layer. Every vertex not in the last layer has two outgoing edges labeled 0 and 1 leading to the next layer. An n-bit input string naturally identifies a path through the graph by reading from left to right. The program accepts or rejects this string depending on whether the path ends at the designated "accept vertex" v_{acc} in the last layer.

Definition 1.2 (Hitting set)**.** *Let $\mathcal{F}$ be a class of functions $f : \{0,1\}^n \to \{0,1\}$. An ϵ-hitting set for $\mathcal{F}$ is a set $H \subseteq \{0,1\}^n$ such that for every $f \in \mathcal{F}$ with $\mathbb{E}[f] \geq \epsilon$, there is some $x \in H$ such that $f(x) = 1$.*

When we say "a log-space hitting set," we mean a family of hitting sets $H_n \subseteq \{0,1\}^n$ such that given input $n \in \mathbb{N}$, the set H_n can be enumerated in space $O(\log n)$. For such a family, $|H_n| \leq \text{poly}(n)$.

Definition 1.5 (Deterministic sampler)**.** *Let $\mathcal{F}$ be a class of functions $f : \{0,1\}^n \to \{0,1\}$. A deterministic ϵ-sampler for $\mathcal{F}$ is a deterministic oracle algorithm A that outputs a real number such that for every $f \in \mathcal{F}$, $|A^f - \mathbb{E}[f]| \leq \epsilon$.*

Let U_n denote the uniform distribution over $\{0,1\}^n$. For two strings x, y, let $x \circ y$ denote the concatenation of x with y. Suppose an ROBP f is clear from context. If u and v are vertices, let $p_{u \to v}$ be the probability that a random walk starting at u reaches v. We use the shorthand $p_{\to v} = p_{v_{\text{start}} \to v}$ and $p_{u \to} = p_{u \to v_{\text{acc}}}$. We use V_i to denote the set of vertices in the i-th layer of the ROBP, where $i \in \{0, 1, \ldots, n\}$.

Theorem 1.7. *Assume that for every constant w, for all $n \in \mathbb{N}$, there is a $\frac{1}{2}$-hitting set for width-w length-n ROBPs that can be computed in space $O(\log n)$. Then for every constant w, for all $n \in \mathbb{N}$ and all $\epsilon > 0$, there is a deterministic ϵ-sampler for width-w length-n ROBPs that runs in space $O(\log(n/\epsilon))$.*

More generally, given a $\frac{1}{2}$-hitting set for width-w' length-n' ROBPs for suitable values $w' = O(w^2)$ and $n' = O(n^2/\epsilon)$, we show how to construct a deterministic ϵ-sampler for width-w length-n ROBPs. If the hitting set has space complexity s, then our deterministic sampler has space complexity $O(s + w \log(n/\epsilon))$. See Theorem 3.1 and Corollary 3.7.

Theorem 1.8 (Unconditional sampler)**.** *For every constant w, for all $n \in \mathbb{N}$ and all $\epsilon > 0$, there is a deterministic ϵ-sampler for width-w length-n ROBPs running in space $O(\log^2 n + \log(1/\epsilon))$.*

3 Deterministic samplers for constant-width ROBPs

In this section, we will show how to use hitting sets to construct deterministic samplers for constant-width ROBPs, thereby proving Theorem 1.7. Most of the work will go toward establishing the following reduction, which is meaningful even for slightly super-constant width.

Theorem 3.1. *Let $w, n \in \mathbb{N}$ and let $\epsilon > 0$. Assume there is an $(\frac{\epsilon}{2n})$-hitting set H for width-$\binom{w}{2} + 1$) length-n ROBPs computable in space s. Then there is a deterministic ϵ-sampler for width-w length-n ROBPs that runs in space $O(s + w \log(n/\epsilon))$.*

Like Theorem 2.1 of the original publication, Theorem 3.1 technically ought to be phrased in terms of families of ROBPs. We should also clarify the model of space-bounded oracle algorithm. We assume that the sampler has write-only access to a "query tape" where it can write down an n-bit query string (the query string does not count against the sampler's space complexity). The sampler can then enter a special "query" state, which returns the result of the query into the algorithm's state and clears the query tape. This natural model was perhaps first studied by Ladner and Lynch [2].

3.1 Setting up the reduction

Toward proving Theorem 3.1, we begin by setting up some notation. For any ROBP f and a string $x \in \{0,1\}^{\leq n}$, let $v_f(x)$ be the vertex reached when f

reads x. Furthermore, define

$$p_f(x) = \mathbb{E}[f(x \circ U_{n-|x|})]\,,$$

i. e., $p_f(x) = p_{v_f(x)\to}$.

Now, let $H \subseteq \{0,1\}^n$ be an ϵ_H-hitting set for width-$(\binom{w}{2}+1)$ ROBPs. For $i \le n$, let H_i be the set of i-bit prefixes of strings in H, i. e., $H_i = \{x_1 x_2 \dots x_i : x \in H\}$. One can verify that H_i is an ϵ_H-hitting set for width-$(\binom{w}{2}+1)$ length-i ROBPs.

Let f be the width-w ROBP to which we have oracle access. Let λ denote the empty string. Our goal is to estimate $p_f(\lambda)$. For each $i \le n$, define an equivalence relation $\sim$ on $\{0,1\}^i$ by the rule

$$x \sim y \iff \forall z \in H_{n-i}, f(x \circ z) = f(y \circ z)\,.$$

Lemma 3.2. *If $x \sim y$, then $|p_f(x) - p_f(y)| < \epsilon_H$.*

Proof. Let $i = |x| = |y|$. Define $g\colon \{0,1\}^{n-i} \to \{0,1\}$ by

$$g(z) = f(x \circ z) \oplus f(y \circ z)\,.$$

The function $g(z)$ can be computed by an ROBP of width $\binom{w}{2}+1$: we have one state in g for each unordered pair of states in f to run the computations $f(x \circ z), f(y \circ z)$ in parallel, along with one additional $\perp$ state in g to indicate that the two computations converged to the same state. If $|p_f(x) - p_f(y)| \ge \epsilon_H$, then H_{n-i} hits g, hence $x \not\sim y$. □

Let $[x]$ denote the equivalence class of x, so $[x] \subseteq \{0,1\}^{|x|}$. Our deterministic sampler will be based on numbers $\widetilde{p}_f([x]) \in [0,1]$ for each equivalence class $[x]$. The definition of $\widetilde{p}_f$ will ensure that $\widetilde{p}_f([x]) \approx p_f(x)$ for typical values of x, although there might be some anomalous values of x where $\widetilde{p}_f([x]) \not\approx p_f(x)$.

The definition of $\widetilde{p}_f([x])$ is inductive. For the base case, when $x \in \{0,1\}^n$, define $\widetilde{p}_f([x]) = f(x)$. This is well-defined, because $x \sim y \implies f(x) = f(y)$. For the inductive step, suppose $x \in \{0,1\}^i$ with $i < n$. Define

$$\widetilde{p}_f([x]) = \max_{x' \in H_i \cap [x]} \left(\frac{1}{2}\widetilde{p}_f([x' \circ 0]) + \frac{1}{2}\widetilde{p}_f([x' \circ 1]) \right)\,, \tag{1}$$

with the convention that $\widetilde{p}_f([x]) = 0$ if $H_i \cap [x] = \varnothing$. Our sampler will output[1] $\widetilde{p}_f([\lambda])$. (In Section 3.3, we will explain in more detail how to compute $\widetilde{p}_f([\lambda])$ in a space-efficient manner.)

3.2 Correctness

The upper bound on $\widetilde{p}_f([x])$ is straightforward:

Claim 3.3. *For every i, for every $x \in \{0,1\}^{n-i}$,*

$$\widetilde{p}_f([x]) \le p_f(x) + i\epsilon_H\,.$$

[1] Actually the sampler's output differs slightly from $\widetilde{p}_f([\lambda])$ due to rounding errors.

Proof. We proceed by induction on i. In the base case $i = 0$, $\widetilde{p}_f([x]) = f(x) = p_f(x)$. For the inductive step $i > 0$, we consider two cases. If $H_i \cap [x] = \varnothing$, then $\widetilde{p}_f([x]) = 0$ and the claim is trivial. Otherwise, there is some $x' \in H_i \cap [x]$ such that

$$\begin{aligned}
\widetilde{p}_f([x]) &= \frac{1}{2}\widetilde{p}_f([x' \circ 0]) + \frac{1}{2}\widetilde{p}_f([x' \circ 1]) && \text{(Equation (1))} \\
&\leq \frac{1}{2}p_f(x' \circ 0) + \frac{1}{2}p_f(x' \circ 1) + (i-1)\epsilon_H && \text{(Induction)} \\
&= p_f(x') + (i-1)\epsilon_H \\
&< p_f(x) + i\epsilon_H && \text{(Lemma 3.2.)}
\end{aligned}$$

□

The lower bound is a little more subtle. If u is a vertex in layer i of f, we say that u is *H-reachable* if there is some $x \in H_i$ with $v_f(x) = u$. Otherwise, we say that u is *H-unreachable*. Let $\widetilde{f}$ be a width-$(w+1)$ ROBP obtained from f by replacing all H-unreachable nodes with reject nodes.[2]

Claim 3.4. *For every i, for every $x \in \{0,1\}^{n-i}$,*

$$\widetilde{p}_f([x]) \geq p_{\widetilde{f}}(x)\,.$$

Proof. We proceed by induction on i. In the base case $i = 0$, $\widetilde{p}_f([x]) = f(x) \geq \widetilde{f}(x) = p_{\widetilde{f}}(x)$. For the inductive step $i > 0$, we consider two cases. If f visits some H-unreachable node when it reads x, then $p_{\widetilde{f}}(x) = 0$ and the claim is trivial. Therefore, assume that when f reads x, every node visited is H-reachable. Then there is some $x' \in H_{n-i}$ such that $v_f(x) = v_f(x')$. Of course when f reads x', every node visited is H-reachable, so

$$v_{\widetilde{f}}(x') = v_f(x') = v_f(x) = v_{\widetilde{f}}(x)\,.$$

Therefore,

$$\begin{aligned}
p_{\widetilde{f}}(x) = p_{\widetilde{f}}(x') &= \frac{1}{2}p_{\widetilde{f}}(x' \circ 0) + \frac{1}{2}p_{\widetilde{f}}(x' \circ 1) \\
&\leq \frac{1}{2}\widetilde{p}_f([x' \circ 0]) + \frac{1}{2}\widetilde{p}_f([x' \circ 1]) && \text{(Induction)} \\
&\leq \widetilde{p}_f(x) && \text{(Equation (1).)}
\end{aligned}$$

(The last inequality uses the fact that $v_f(x) = v_f(x')$ and hence $x \sim x'$.) □

Corollary 3.5. $|\widetilde{p}_f([\lambda]) - \mathbb{E}[f]| \leq n \cdot \epsilon_H$.

Proof. By Claim 3.3,

$$\widetilde{p}_f([\lambda]) \leq p_f(\lambda) + n \cdot \epsilon_H = \mathbb{E}[f] + n \cdot \epsilon_H\,.$$

In the other direction, by Claim 3.4,

$$\widetilde{p}_f([\lambda]) \geq p_{\widetilde{f}}(\lambda) = \mathbb{E}[\widetilde{f}]\,.$$

[2]More precisely, add an extra node to each layer labeled $\perp$. In layers prior to the final layer, both outgoing edges from $\perp$ lead to $\perp$, and both outgoing edges from H-unreachable nodes lead to $\perp$. In the final layer, H-unreachable nodes and $\perp$ are reject nodes.

Define $g\colon \{0,1\}^n \to \{0,1\}$ by

$$g(x) = 1 \iff \text{when } f \text{ reads } x, \text{ an } H\text{-unreachable node is visited.}$$

Then g can be computed by a width-$(w+1)$ ROBP by a construction very similar to that of $\widetilde{f}$. By construction, g rejects every string in H. Therefore, $\mathbb{E}[g] < \epsilon_H$. Furthermore, $g(x) = 0 \implies f(x) = \widetilde{f}(x)$. Therefore, $|\mathbb{E}[\widetilde{f}] - \mathbb{E}[f]| < \epsilon_H$, so $\widetilde{p}_f([\lambda]) > \mathbb{E}[f] - \epsilon_H$. □

3.3 Efficiently computing $\widetilde{p}_f([\lambda])$

To complete the proof of Theorem 3.1, we just need to show how to efficiently compute $\widetilde{p}_f([\lambda])$. This is fairly straightforward from the definitions; the details follow.

Proof of Theorem 3.1. Say a string $x \in H_i$ is a *representative* if it is the lexicographically first element of $[x] \cap H_i$. Let $x^{(i,1)}, x^{(i,2)}, \ldots$ be an enumeration of the representatives in H_i in lexicographic order. Given i, j, and oracle access to f, one can compute $x^{(i,j)}$ in space $O(s)$.

Our sampler works its way backward through the branching program, starting at layer n and ending with layer 0. The sampler stores data about layer i and uses it when processing layer $i-1$. Specifically, the data stored regarding layer i consists of a list of numbers $p_{i,1}, p_{i,2}, \ldots$, with the interpretation $p_{i,j} = \widetilde{p}_f([x^{(i,j)}])$, or rather $p_{i,j} \approx \widetilde{p}_f([x^{(i,j)}])$ due to rounding error.

For layer n, we can compute this value exactly by setting $p_{i,j} = f(x^{(i,j)})$. Given these values for layer $i+1$, define $\overline{p}_f(x)$ for each $x \in \{0,1\}^{i+1}$ by

$$\overline{p}_f(x) = \begin{cases} p_{i+1,j} & \text{if } j \text{ is such that } x^{(i+1,j)} \sim x \\ 0 & \text{if no such } j \text{ exists.} \end{cases}$$

(Note that j is unique if it exists, and note that $\widetilde{p}_f([x]) = 0$ if such a j does not exist.) Then, we compute each value $p_{i,j}$ by the rule

$$p_{i,j} := \max_{x' \in H_i \cap [x^{(i,j)}]} \left(\frac{1}{2}\overline{p}_f(x' \circ 0) + \frac{1}{2}\overline{p}_f(x' \circ 1) \right), \tag{2}$$

with the convention that $p_{i,j} = 0$ if $H_i \cap [x^{(i,j)}] = \varnothing$.

The sampler performs the arithmetic in Equation (2) to within $\lceil \log(2n/\epsilon) \rceil$ bits of precision. This ensures that the rounding error is not too large in each step; by induction, $|p_{i,j} - \widetilde{p}_f([x^{(i,j)}])| \le \frac{\epsilon(n-i)}{2n}$. The sampler outputs $p_{0,1}$, which is within ϵ of $\mathbb{E}[f]$ by Corollary 3.5, since $\epsilon_H = \frac{\epsilon}{2n}$.

The number of vertices in each layer of f is at most w, so the number of equivalence classes in $\{0,1\}^i$ is also at most w. Therefore, there are at most w representatives in H_i, and hence there are only w numbers $p_{i,j}$ being stored for each layer. Storing those numbers for the layer currently being processed and the layer most recently processed takes $O(w \log(n/\epsilon))$ bits of space, so overall, the space complexity of the sampler is $O(s + w \log(n/\epsilon))$ as claimed. □

Interestingly, the sampler of Theorem 3.1 can be implemented to be nonadaptive, because it only queries f at strings of the form $x \circ y$ or $x \circ b \circ z$, where $x \in H_i$, $y \in H_{n-i}$, $b \in \{0,1\}$, and $z \in H_{n-i-1}$.

3.4 Applying the reduction

Proof of Theorem 1.8. Hoza and Zuckerman constructed an $\left(\frac{\epsilon}{2n}\right)$-hitting set H even for *polynomial*-width ROBPs that can be computed in space $O(\log^2 n + \log(1/\epsilon))$ [1]. Combining this result with Theorem 3.1 immediately proves Theorem 1.8. □

To prove Theorem 1.7, we must first amplify the assumed $\frac{1}{2}$-hitting set to get an $\left(\frac{\epsilon}{2n}\right)$-hitting set. This is straightforward, although we must pay a small penalty in terms of width, length, and cardinality.

Lemma 3.6. *Suppose H is a $\frac{1}{2}$-hitting set for width-$(w+1)$ length-(nm) ROBPs. Divide each string $x \in H$ into blocks of length n, $x = x^{(1)} \circ x^{(2)} \circ \cdots \circ x^{(m)}$. Let $H' = \{x^{(i)} : x \in H, i \in [m]\}$. Then H' is a $(\frac{1}{m})$-hitting set for width-w length-n ROBPs.*

Proof. Let f be a width-w length-n ROBP with $\mathbb{E}[f] \geq 1/m$. Define $g\colon (\{0,1\}^n)^m \to \{0,1\}$ by

$$g(x^{(1)} \circ \cdots \circ x^{(m)}) = \bigvee_{i \in [m]} f(x^{(i)}) \,.$$

Then g can be computed by a width-$(w+1)$ ROBP. Furthermore,

$$\mathbb{E}[g] = 1 - (1 - \mathbb{E}[f])^m \geq 1 - \left(1 - \frac{1}{m}\right)^m > \frac{1}{2} \,.$$

Therefore, H hits g, hence H' hits f. □

Combining Lemma 3.6 with Theorem 3.1 proves the following corollary, which immediately implies Theorem 1.7.

Corollary 3.7. *Let $w, n \in \mathbb{N}$ and let $\epsilon > 0$. Let $w' = \binom{w}{2} + 2$ and $n' = n \cdot \lceil 2n/\epsilon \rceil$. Assume there is a $\frac{1}{2}$-hitting set H for width-w' length-n' ROBPs computable in space s. Then there is a deterministic ϵ-sampler for width-w length-n ROBPs that runs in space $O(s + w \log(n/\epsilon))$.*

References

[1] William M. Hoza and David Zuckerman: Simple optimal hitting sets for small-success **RL**. *SIAM J. Comput.*, 49(4):811–820, 2020. Preliminary version in FOCS'18. [doi:10.1137/19M1268707, ECCC TR18-063]

[2] Richard E. Ladner and Nancy A. Lynch: Relativization of questions about log space computability. *Math. Sys. Theory*, 10(1):19–32, 1976. [doi:10.1007/BF01683260]